

SkyInsight

Best Model Descriptions & Strengths

Technical Validation Document · 2026

This document describes the winning model for each SkyInsight module, its architecture, key hyperparameters, and why it outperforms alternatives.

#	Module	Best Model
1	Cabin Cleanliness Detection (CV)	EfficientNetV2S + GeM + Focal Loss
2	Crowd Analytics (CV)	EfficientNetV2S + GeM + Focal Loss
3	Baggage Handling QC (CV)	DenseNet121 + GeM + Focal Loss
4	Weather Delay Prediction	BatchNorm DNN (4-block, AdamW + Cosine LR)
5	Route Profitability — Regression	LightGBM Regressor + SHAP
6	Route Profitability — Classification	LightGBM / Extra Trees Classifier
7	Customer Churn Prediction	Random Forest Classifier (time-aware)
8	Customer Satisfaction	XGBoost / LightGBM Classifier
9	CO ₂ Emissions Forecasting	Random Forest + Ensemble (RF + XGB + LGB)
10	NLP — Sentiment Analysis	DistilBERT (Transformer) + VADER dual-stack
11	NLP — Topic Modeling	LDA (8 topics, CountVectorizer)

Cabin Cleanliness Detection

Model: EfficientNetV2S with GeM Pooling

Binary image classifier distinguishing Clean from Dirty cabin states. EfficientNetV2S is a compound-scaled convolutional network that balances depth, width, and resolution simultaneously — making it the most parameter-efficient choice for high-accuracy image classification.

Why EfficientNetV2S wins over alternatives
vs EfficientNetB0: V2S uses Fused-MBConv blocks in early stages — faster training and +2–4% accuracy.
vs ResNet50V2: V2S achieves higher accuracy with fewer parameters (20M vs 25M).
vs ConvNeXtTiny: V2S has a more stable training curve for small aviation datasets.
vs DenseNet121: V2S scales better with fine-tuning on domain-specific imagery.

Key Hyperparameters

Parameter	Value	Reason
Image size	224 × 224	Standard ImageNet resolution — strong pretrained features
Head LR	1e-3	Fast convergence of new classification layers
Fine-tune LR	5e-6	Very slow to preserve pretrained features
Unfreeze ratio	40%	Bottom 60% frozen — only top layers re-trained
Dropout (1st)	0.45	High dropout — cabin data can be visually similar
Dropout (2nd)	0.30	Graduated decrease toward output
Focal gamma	2.5	Strong down-weighting of easy negatives
Label smoothing	0.05	Prevents overconfident softmax predictions
Mixup alpha	0.2	Mild interpolation — preserves class boundaries
Head epochs	20	Train head fully before unfreezing backbone
Fine-tune epochs	15	Short fine-tuning to avoid forgetting
Batch size	32	Memory-efficient while giving stable gradients

Key Strengths

- ▶ Focal Loss ($\gamma=2.5$): aggressively focuses training on hard, misclassified samples — critical when dirty cabins appear in many forms but clean ones are visually similar
- ▶ GeM Pooling: aggregates spatial features with a learnable exponent p , retaining texture patterns (stains, clutter) that standard GAP would average out
- ▶ 40% backbone unfreezing: preserves low-level texture detectors while adapting high-level semantics to airline cabin imagery
- ▶ TTA $\times 5$: at inference, 5 augmented views are averaged — reduces sensitivity to camera angle and lighting variation
- ▶ Temperature Scaling: post-hoc probability calibration ensures the model's stated confidence is statistically honest
- ▶ MC-Dropout $\times 15$: runs 15 stochastic forward passes to produce per-class uncertainty bands — flags ambiguous cabin states

2 Crowd Analytics

Model: EfficientNetV2S with GeM Pooling

Binary classifier distinguishing Low Crowd from High Crowd scenes in airport gates and boarding areas.

Why EfficientNetV2S wins for crowd scenes

Crowd detection requires global scene understanding (density, spacing) — EfficientNetV2S's compound scaling captures both fine and coarse patterns.

The relatively narrow augmentation config (rotation $\pm 5^\circ$, zoom $\pm 10\%$) matches real operational camera conditions — less distortion than cabin images.

Crowd scenes are less visually ambiguous than cleanliness → lighter augmentation = less variance introduced during training.

Key Hyperparameters

Parameter	Value	Reason
Head LR	8e-4	Slightly lower than cleanliness — crowd features transfer better
Fine-tune LR	5e-6	Same conservative rate for backbone
Unfreeze ratio	25%	Only top 25% unfrozen — pretrained crowd-scene features are very good
Dropout (1st)	0.40	Moderate — crowd classes are more distinct than cleanliness
Focal gamma	2.0	Lighter focus than cleanliness — class balance is more even
Head epochs	22	Slightly more head training — needed for new pooling head
Fine-tune epochs	10	Shorter fine-tuning — backbone generalises well
Horizontal flip	False	Crowd density is invariant to direction — flip unnecessary

Key Strengths

- ▶ GeM Pooling: for crowd scenes, the learnable exponent p selects dominant density patterns rather than averaging all spatial locations
- ▶ Minimal unfreeze (25%): pretrained ImageNet features already encode crowd-like patterns (people, furniture density) — less re-training needed
- ▶ Conservative augmentation: only mild brightness and slight zoom — protects the spatial density signal that defines the class
- ▶ Same TTA/MC-Dropout/Temperature/Rejection pipeline as other CNN models — consistent, production-ready prediction stack

3 Baggage Handling QC

Model: DenseNet121 with GeM Pooling

Binary classifier: Good Luggage vs Damaged Luggage. DenseNet121 was specifically selected over EfficientNetV2S for this task due to the nature of damage detection.

Why DenseNet121 wins for damage detection

DenseNet connects every layer to every subsequent layer — feature reuse is maximal. Damage manifests as subtle local texture anomalies (tears, dents, discolouration).

These fine-grained differences are preserved throughout all layers rather than being progressively discarded as in standard CNNs.

DenseNet is historically strongest on medical and defect-detection tasks — baggage QC shares the same 'find a small anomaly' structure.

vs EfficientNetV2S: V2S is superior for global scene classification; DenseNet121 is superior for local defect/anomaly patterns.

Key Hyperparameters

Parameter	Value	Reason
Backbone	DenseNet121	Dense connectivity for fine-grained defect detection
Head LR	8e-4	Conservative — damage features are subtle
Fine-tune LR	8e-6	Slightly higher than others — DenseNet needs more adaptation
Head epochs	16	Moderate head training
Fine-tune epochs	12	Extra fine-tuning for domain adaptation
Focal gamma	variable	Configured per class imbalance level
GeM exponent	learnable	Adaptive pooling to highlight damage texture

Key Strengths

- ▶ Dense connectivity: gradients flow directly from output to every preceding layer — no vanishing gradient, every feature is accessible
- ▶ Feature reuse at inference: subtle damage textures captured in shallow layers are directly concatenated into the deep classification decision
- ▶ GeM Pooling: the learnable p aggregates the spatially localised damage signals — damaged bags have texture anomalies in small regions, not everywhere
- ▶ Focal Loss: critically important here as 'good luggage' images are often far more common than damaged ones in real operations
- ▶ Strong augmentation: handles varied conveyor belt lighting, camera angles, and orientation — real baggage arrives in any configuration

4 Weather Delay Prediction

Model: Batch-Normalised Deep Neural Network

Binary classification on tabular flight statistics. Predicts WDCase = 1 if weather delay > 100 minutes. This is a custom-architected DNN, not a pre-trained model — designed from scratch for tabular airline data.

Architecture: 4-Block BatchNorm DNN

Input → Dense(256) → BatchNorm → ReLU → Dropout(0.3) + L2(1e-4)
→ Dense(128) → BatchNorm → ReLU → Dropout(0.2) + L2(1e-4)
→ Dense(64) → BatchNorm → ReLU → Dropout(0.2) + L2(1e-4)
→ Dense(32) → BatchNorm → ReLU → Dropout(0.1) + L2(1e-4)
→ Dense(1) → Sigmoid (binary probability output)

Why this DNN wins over tree-based models for this task

The dataset has high-volume flight statistics with complex non-linear interaction patterns — DNN captures higher-order relationships.
BatchNorm stabilises training across a wide range of feature magnitudes (flight counts vs delay minutes differ by orders of magnitude).
Class weights compensate for severe imbalance (weather delays > 100 min are rare) — tree models struggle more with this on large tabular data.
AUC-ROC = 0.9991 on test set — effectively perfect discrimination between severe and non-severe weather delay events.

Training Configuration

Component	Value / Config	Why it matters
Optimiser	AdamW ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-7$)	Decoupled weight decay = better generalisation than L2 in Adam
LR schedule	Cosine Decay: $3e-3 \rightarrow 1e-5$	Smooth LR reduction — no sudden drops, better final convergence
Warm-up	10 epochs ($0 \rightarrow \text{initial_lr}$)	Prevents instability in early training when weights are random
Weight decay	$1e-4$	Decoupled from gradient — prevents large weights without distorting Adam steps
Batch size	4096	Large batch $\rightarrow$ lower gradient noise on big tabular datasets
Max epochs	150 (early stop on val_AUC)	Trains to optimal AUC, not just accuracy
Dropout schedule	$0.3 \rightarrow 0.2 \rightarrow 0.2 \rightarrow 0.1$	Progressive decrease — less regularisation near output
L2 penalty	$1e-4$ on all Dense layers	Second regulariser to complement Dropout
Split	64% train / 16% val / 20% test	Validation used for EarlyStopping, test is untouched
StandardScaler	Fit on train only	Eliminates leakage — val/test scaled with train statistics
Class weights	sklearn compute_class_weight balanced	Penalises false negatives on rare severe-delay events

Results vs Targets

Metric	Score	Target
Accuracy	99.91%	≥ 95% 
AUC-ROC	≈ 0.9999	≥ 0.98 
Precision	≈ 100%	≥ 90% 
Recall	≈ 99.9%	≥ 85% 
F1-Score	≈ 1.00	≥ 90% 
Avg Precision	≈ 1.00	≥ 0.90 

Key Strengths

- ▶ BatchNorm after every Dense layer: normalises activations to unit variance — eliminates internal covariate shift, allows 3–5× higher learning rate
- ▶ Progressive Dropout: heavier regularisation early (0.3), lighter near output (0.1) — prevents memorisation without crippling the final decision layer
- ▶ AUC-ROC as primary training signal: monitors discrimination ability rather than raw accuracy — essential for the imbalanced delay dataset
- ▶ Cosine + Warm-up LR: warm-up prevents catastrophic early updates; cosine decay gives a smooth 'cool-down' rather than aggressive plateau behaviour
- ▶ Large batch (4096): tabular data with many near-identical records benefits from large batches to see sufficient diversity per update

5 Route Profitability — Regression

Model: LightGBM Regressor

Predicts Profit_Margin (%) from pre-flight operational features across 7,974 flight records on 30 routes. LightGBM was the top performer in a 7-model benchmark evaluated by 5-fold cross-validation + holdout test.

Configuration: LightGBM Regressor
n_estimators=500 | max_depth=6 | learning_rate=0.05
num_leaves=63 | subsample=0.8 | colsample_bytree=0.8
n_jobs=-1 | random_state=RANDOM_STATE

Why LightGBM wins over the other 6 models
vs Ridge / Lasso / ElasticNet: profit margin has highly non-linear relationships with load factor and seasonality — linear models cannot capture these.
vs Random Forest (300 trees): LightGBM uses leaf-wise tree growth vs level-wise → finds deeper, more accurate splits on the same data.
vs Extra Trees: faster with better regularisation via learning_rate + subsample; Extra Trees use random splits, LightGBM uses gradient-optimised ones.
vs Gradient Boosting (sklearn): LightGBM is 10–100× faster and uses histogram-based splits — critical for 7,974 records × 500 trees.
SHAP compatibility: TreeExplainer produces exact Shapley values for LightGBM in seconds — perfect for route-level interpretability.

Features Used

Feature	Business Meaning
Aircraft_Capacity	Fixed-cost dilution potential — more seats = lower CASK if filled
Passengers / Load_Factor	Primary demand signal — strongest predictor of margin
Flight_Hours	Distance proxy — drives fuel, crew, and depreciation cost
Month / Quarter / Season	Captures seasonality (peak Dec–Feb vs loss-making Jun–Aug)
Aircraft_Type	Encodes efficiency differences (787-9 vs A320)
Route_Category	Short / Medium / Long-haul structural economics
Demand_Level	Categorical demand intensity at time of flight
Destination	Route-specific structural factors (airport fees, competition)

Results

- Test $R^2 = 0.68$ (model explains 68% of profit margin variance)
- RMSE ≈ 21.7 percentage points
- MAE ≈ 16.1 percentage points
- Top SHAP drivers: Load_Factor and Flight_Hours (confirmed via dependence plots)
- Use case: operational planning tool, not final financial authority

6 Route Profitability — Classification

Model: LightGBM / Extra Trees Classifier

Binary task: classify each flight as Profitable (margin > 0%) or Loss-Making. Same 7-model benchmark as regression, evaluated with Accuracy, Precision, Recall, F1, and AUC-ROC.

Configuration

- ▶ Same feature set as regression (Aircraft_Capacity, Load_Factor, Flight_Hours, seasonality, route category, aircraft type)
- ▶ LightGBM Classifier with same hyperparameters as regression (n_estimators=500, max_depth=6, lr=0.05)
- ▶ StratifiedKFold (5-fold) preserves class ratio in each fold

Key Strengths

- ▶ Gradient-optimised splits: directly minimises classification loss on margin sign rather than magnitude — sharper decision boundary than regression models
- ▶ Low learning rate (0.05) + high n_estimators (500): ensemble of many weak learners each correcting the residual of the previous — lower variance than a single deep tree
- ▶ subsample=0.8: stochastic row sampling per tree reduces overfitting and improves generalisation on route-level patterns

7 Customer Churn Prediction

Model: Random Forest Classifier

Time-series binary classification on monthly loyalty data. Churn is defined as zero flights for the next 6 consecutive months. The dataset requires time-aware engineering and a time-based split (no random shuffling).

Configuration: Random Forest Classifier
n_estimators=400 | max_depth=None | min_samples_leaf=2
class_weight='balanced_subsample' | random_state=RANDOM_STATE | n_jobs=-1

Why Random Forest wins over Logistic Regression and XGBoost here
vs Logistic Regression: churn drivers (recency × frequency interactions, slope of engagement trend) are non-linear — LR cannot capture these without manual polynomial features.
vs XGBoost: RF with n_estimators=400 and balanced_subsample handles the severe class imbalance more robustly here; XGBoost requires careful scale_pos_weight tuning.
balanced_subsample: each tree gets a class-balanced bootstrap sample — no individual tree dominates the majority class.
max_depth=None: allows trees to grow deep enough to capture 6-month rolling patterns and interaction effects between recency, frequency, and redemption.
Deployed via skl2onnx for ONNX export — production-ready for Power BI and real-time scoring.

Feature Engineering (Time-Aware)

Feature Block	Features Generated
Recency	months_since_last_flight — most powerful churn predictor
Frequency	rolling_avg_flights_3m, rolling_avg_flights_6m
Trend	rolling_slope_flights (polyfit deg=1), rolling_slope_points
Engagement	cumulative_points, redemption_ratio (points redeemed / earned)
Drop signal	ratio of recent 3m flights vs previous 3m — activity collapse indicator
Aggregates	lifetime_flights, avg_monthly_distance, total_months_active

Key Strengths

- Time-based split: training on earliest 80%, testing on most recent 20% — mirrors real-world deployment where you predict future churn from past behaviour
- min_samples_leaf=2: prevents single-sample leaf nodes that memorise noise in sparse monthly data
- Feature selection via RF importance: pre-filters features before final model — removes noisy columns that would degrade OOB performance
- High-risk customer ranking: outputs a scored list sorted by churn probability — actionable for CRM retention campaigns
- Reported AUC-ROC = 0.923, F1 = 0.918 — flagged for immediate production deployment

8 Customer Satisfaction Prediction

Model: XGBoost Classifier (LightGBM fallback)

Multi-class or binary classification on passenger survey data. The pipeline is schema-adaptive — it detects the target automatically and adjusts the objective function accordingly.

Configuration: XGBoost Classifier
n_estimators=500 | max_depth=6 | learning_rate=0.05
subsample=0.9 | colsample_bytree=0.9
objective='multi:softprob' (multi-class) or 'binary:logistic' (binary)
eval_metric='mlogloss' or 'logloss'

Why XGBoost wins over Random Forest and Logistic Regression
vs Logistic Regression: satisfaction is driven by non-linear combinations of service ratings, delays, and traveller type — LR with saga solver converges but misses interactions.
vs Random Forest (500 trees): XGBoost's gradient boosting sequentially corrects errors — lower bias than RF's parallel averaging on structured survey data.
colsample_bytree=0.9: random feature sampling per tree prevents correlated trees and reduces overfitting on survey scale items (which are often correlated).
Schema-adaptive objective: automatically switches between multi:softprob (multiple satisfaction levels) and binary:logistic — handles any survey schema.

Evaluation Metrics Used

Metric	Purpose
Accuracy / Balanced Accuracy	Overall and class-balanced correctness
Precision / Recall / F1	Standard trio (weighted avg for multi-class)
ROC-AUC / PR-AUC	Discrimination and precision-recall trade-off
Matthews Correlation Coef.	Most robust single metric for imbalanced classes
Cohen's Kappa	Agreement measure corrected for chance
Log Loss	Probabilistic calibration quality
CV (StratifiedKFold, 5-fold)	Training-set validation to guard against overfitting

Key Strengths

- 7-metric evaluation: most rigorous evaluation in the project — no single metric can mislead on this complex survey dataset
- SHAP explainability: top-10 features identified, delay-related features explicitly flagged — tells the business which service attributes drive satisfaction most
- Delay impact sub-analysis: separate analysis quantifying the satisfaction loss per minute of departure/arrival delay
- KMeans segmentation (k=4): four customer profiles built on the selected feature space — each cluster characterised by satisfaction rate and churn-proxy
- Winner selected by F1 score — the most balanced metric when satisfaction classes are not equally distributed

9 CO₂ Emissions Forecasting

Model: Random Forest + Averaging Ensemble (RF + XGB + LGB)

Time-series regression predicting annual CO₂ emissions per country. The final deliverable is a 10-year iterative forecast. The Random Forest is the primary export model; the ensemble is used for uncertainty estimation and robustness.

Configuration: Random Forest Regressor (primary)

n_estimators=300+ | max_depth=6–15 (tuned per country)

random_state=42 | n_jobs=-1

Ensemble: simple average of RF + XGBoost + LightGBM predictions

Why Random Forest + Ensemble wins for time-series emissions forecasting

vs Linear / Lasso / ElasticNet: emissions trend is non-linear (COVID dip, recovery surge, policy changes) — linear models cannot capture these structural breaks.

vs SVR: SVR struggles with the lag/rolling feature structure and requires careful kernel tuning per country.

vs KNN: distance-based methods fail when temporal order matters — two years with similar lag values may have completely different trend momentum.

vs Prophet alone: Prophet fits trend + seasonality but cannot use the rich engineered lag features (Lag1/2/3, momentum, rolling std) that drive the RF.

Ensemble advantage: averaging 3 gradient-based models (RF + XGB + LGB) cancels individual model biases — lower variance on the 10-year horizon.

OOS R² is used instead of standard R² — correct for trending time series where the naive baseline (last known value) is the real benchmark.

Temporal Features (13 engineered)

Feature	What it captures
Emissions_Lag1/2/3	Direct autoregression — last 1, 2, 3 years of emissions
YoY_Growth	Year-over-year percentage change
Growth_Lag1	Lagged growth rate — momentum of acceleration/deceleration
Rolling_3Y_Avg	Short-term trend smoothing
Rolling_5Y_Avg	Medium-term trend smoothing
Rolling_3Y_Std	Volatility of recent emissions — measures stability
Momentum_2Y	2-year directional momentum (rising / falling signal)
Log_Lag1	Log-transformed lag — handles exponential growth patterns
Year / Year_Idx / Year_Sq	Time index and quadratic time component

Key Strengths

- ▶ Iterative forecasting: each predicted year is fed back as input for the next — the model self-conditions on its own predictions for 10 years ahead
- ▶ OOS R²: avoids the misleading negative R² that sklearn's standard R² produces on trending series — honest benchmark against the naive forecast
- ▶ 10-model comparison: all models visualised on same test-year chart — transparent selection of the winner
- ▶ Multi-country: RF applied to top 8 emitting countries individually — country-specific RMSE computed for regional granularity
- ▶ Anomaly detection: Isolation Forest flags structural breaks (e.g. COVID 2020) that the model should not extrapolate

- ONNX export: RF serialised for production inference in Power BI and Azure ML

10 NLP Sentiment Analysis

Model: DistilBERT (Primary) + VADER (Secondary)

Dual-stack sentiment system applied to a union of two airline review datasets. Every review receives both a VADER compound score and a DistilBERT label — enabling agreement analysis and handling reviews where the two methods diverge.

Why a dual-stack (not just one model)?

VADER is fast, interpretable, and requires no GPU — runs on 100% of rows with no truncation.

DistilBERT is contextual — understands negation, sarcasm, and multi-clause reviews that confuse lexicon-based VADER.

Divergence between the two flags genuinely ambiguous reviews — a key quality signal for the airline.

VADER gives a continuous score; DistilBERT gives a categorical label — complementary information types.

DistilBERT (Primary — Transformer)

Aspect	Detail
Model	distilbert-base-uncased-finetuned-sst-2-english (HuggingFace Hub)
Architecture	6-layer distilled BERT — 40% smaller, 60% faster than BERT-base, 97% of BERT accuracy
Fine-tuning	Pre-trained on SST-2 (Stanford Sentiment Treebank) — strong English sentiment baseline
Input	Tokenised review text, max 512 tokens (truncated)
Device	GPU if CUDA available (DEVICE=0), else CPU (DEVICE=-1)
Output	POSITIVE / NEGATIVE label + confidence score
Why DistilBERT over BERT	Same accuracy for sentiment, 2× faster — practical for bulk review processing
Why over RoBERTa	Pre-fine-tuned SST-2 checkpoint available directly — no additional training required

VADER (Secondary — Lexicon-Based)

Aspect	Detail
Library	NLTK SentimentIntensityAnalyzer (VADER lexicon)
Output	Compound score ∈ [-1, 1] → binned: Negative / Neutral / Positive
Strength	Domain-specific rules for punctuation (!!!), capitalisation, and emoji
Speed	No GPU required — processes all rows instantly
Limitation	Lexicon-based: cannot understand context, negation in complex sentences

Key Strengths

- ▶ Contextual understanding (DistilBERT): 'not bad at all' is positive — VADER would score this ambiguously
- ▶ Full coverage (VADER): every single review gets a score with no GPU requirement — no data skipped
- ▶ Agreement rate tracking: reviews where VADER and DistilBERT disagree are flagged — high-value ambiguous cases for manual review
- ▶ Continuous + categorical outputs: VADER compound score enables ranking

11 NLP Topic Modeling

Model: Latent Dirichlet Allocation (LDA)

Unsupervised topic extraction from preprocessed airline reviews. LDA discovers 8 latent topics — each review is assigned to its dominant topic for downstream segmentation and reporting.

Configuration: LDA

n_components=8 (topics) | random_state=42
CountVectorizer: max_features=3000 | stop_words='english'
Topic assignment: argmax of per-document topic distribution

Why LDA over NMF and BERTopic for this task

vs NMF (Non-negative Matrix Factorisation): LDA has a probabilistic generative model — produces interpretable per-word topic probabilities, not just soft clusters.

vs BERTopic: BERTopic requires SBERT embeddings and clustering — significant compute overhead for a dataset where 8 interpretable topics is the deliverable.

vs LSA / SVD: LDA explicitly models documents as mixtures of topics — more natural for reviews that discuss multiple service aspects.

LDA top-terms per topic are directly usable in Power BI dashboards — no post-processing needed.

Pipeline

Step	Detail
Text preparation	text_nostop — cleaned, lowercased, stopword-removed review text
Vectorisation	CountVectorizer (max_features=3000, stop_words='english') → Document-Term Matrix
LDA fitting	lda.fit_transform(dtm) — returns document × topic probability matrix
Topic assignment	df['topic_id'] = topic_matrix.argmax(axis=1)
Interpretation	Top-N terms per topic extracted from lda.components_

Key Strengths

- ▶ Soft assignment: each review has a full probability distribution over all 8 topics — not just a hard label
- ▶ Interpretable top-terms: each topic's defining words are directly readable (e.g. 'seat', 'legroom', 'comfort' → seating experience topic)
- ▶ 8 topics: empirically chosen to balance granularity (enough topics to be actionable) vs coherence (not so many they overlap)
- ▶ Count vectoriser: raw term frequency rather than TF-IDF — LDA's probabilistic model handles term weighting internally
- ▶ Integration: topic_id column written back to the main dataframe → directly joinable with sentiment scores for cross-analysis